ПР6. Генерация и рефакторинг кода с помощью диаграмм

Диаграммы классов в Visual Studio

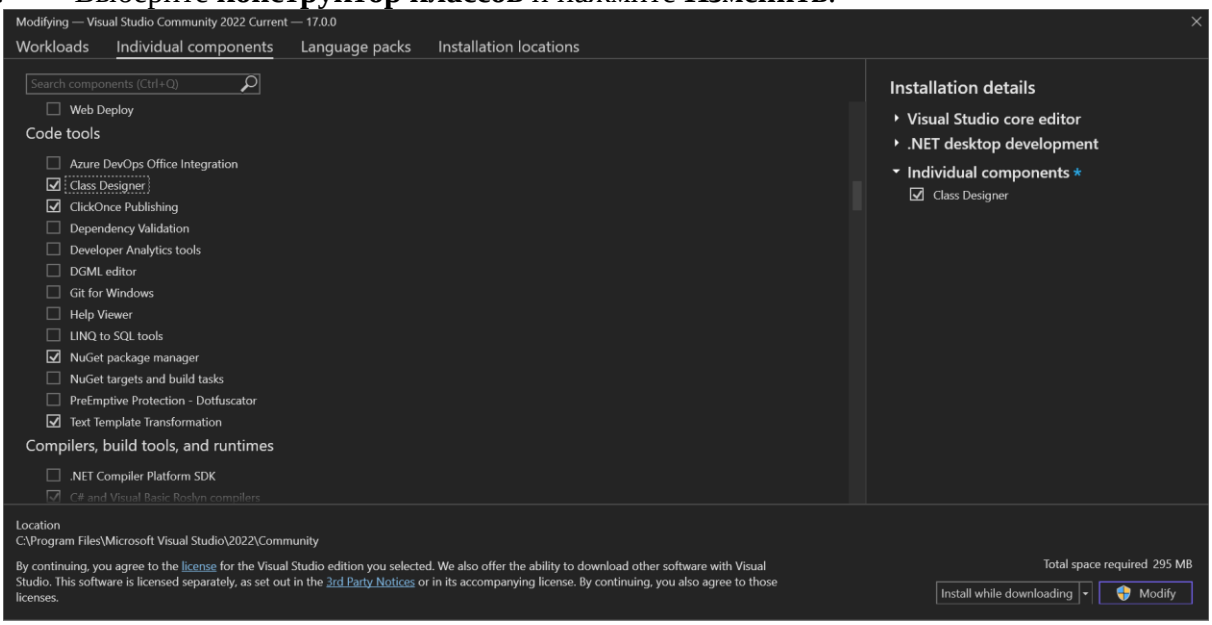
Традиционно моделирование программного обеспечения осуществляется отдельно от кодирования. Как правило, это происходит на этапе проектирования, который заканчивается до начала кодирования. В большей части случаев диаграммы моделирования, построенные на этапе проектирования, быстро устаревают в ходе разработки программного обеспечения и теряют свое значение при неизбежном изменении проекта.

Утилита Class Designer системы Visual Studio позволяет внедрить моделирование в интегрированную среду разработки в виде действий, которые можно выполнить в любое время на всем протяжении разработки проекта. Диаграммы классов динамически конструируются по исходному коду, т.е. они никогда не устаревают. Любые изменения, внесенные в исходном коде, немедленно отражаются на диаграмме классов, а любое изменение диаграммы классов отражается в коде.

Установка компонента конструктора классов

Для установки компонента конструктора классов, выполните следующие действия:

1. Откройте **Visual Studio Installer** в меню "Пуск" Windows или выберите **Средства**>**Получить средства и компоненты** на панели меню в Visual Studio. **открывается** установщик Visual Studio.
2. В поле Visual Studio, нажмите кнопку **Изменить** в меню параметров.
3. Выберите вкладку **Отдельные компоненты**, а затем прокрутите вниз до категории **Инструменты кода**.
4. Выберите **конструктор классов** и нажмите **Изменить**.

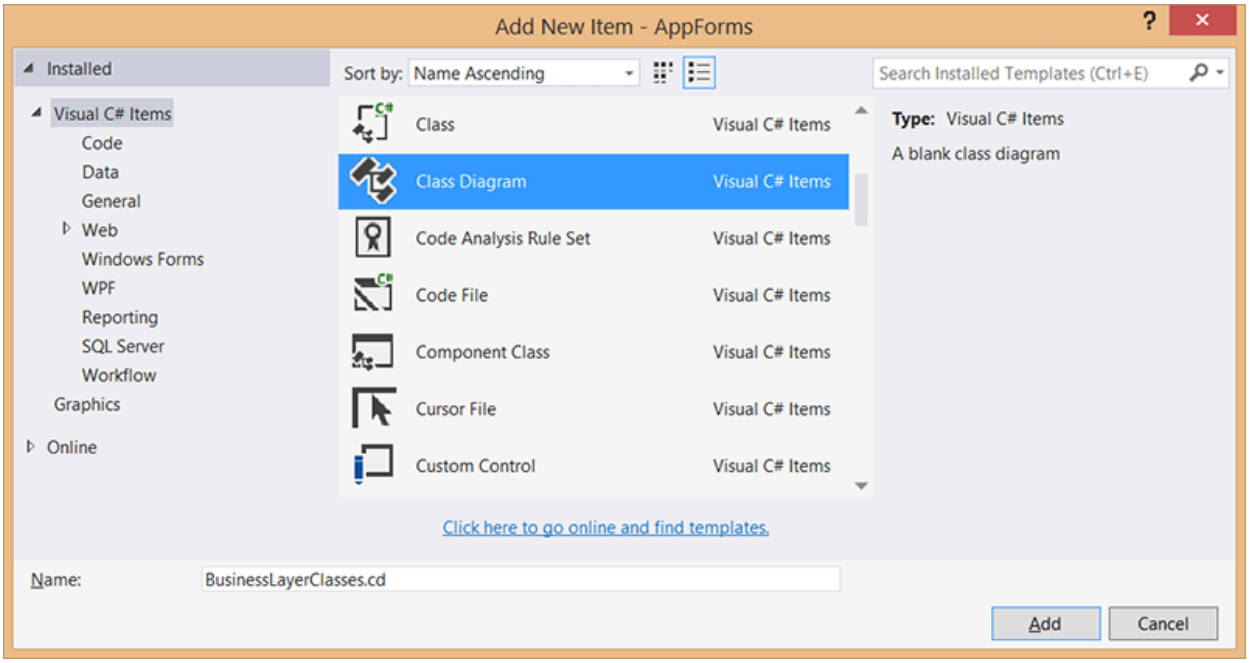


Компонент **конструктора классов** начнет установку.

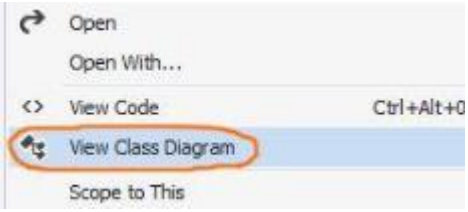
Создание диаграммы классов

Процесс проектирования приложения обычно подразумевает как минимум наличие схемы классов, которые будут созданы, и описание их взаимодействия. Среда Visual Studio имеет область проектирования Class Designer, в которой можно формировать диаграмму классов. Затем в классы можно добавлять поля, свойства и методы, а также устанавливать взаимоотношения между ними.

Несмотря на то, что этот объект называется диаграммой классов, он поддерживает классы, структуры, перечисления, интерфейсы, абстрактные классы и делегаты. Существует несколько способов, с помощью которых разработчик может добавлять диаграмму классов в свой проект. Например, он может открыть диалоговое окно Add New Item, чтобы создать новую пустую диаграмму классов в проекте:



Новую диаграмму классов можно добавить в проект, щелкнув на кнопке View Class Diagram инструментальной панели в окне Solution Explorer или щелкнув правой кнопкой мыши на проекте или классе, а затем выбрав команду View Class Diagram в контекстном меню. Если при этом проект был выделен, то система Visual Studio автоматически добавит все типы, определенные в проекте, в начальную диаграмму классов. Хотя в некоторых случаях это может оказаться удобным, для проекта, содержащего большое количество классов, процесс создания и уточнения такой диаграммы может оказаться слишком долгим.



В отличие от некоторых программных средств, требующих, чтобы все типы, существующие в проекте, принадлежали одной и той же диаграмме классов, в системе Visual Studio диаграмма классов может содержать сколько угодно пользовательских классов. Это позволяет добавлять в одно решение несколько диаграмм классов. Область действия утилиты Class Designer ограничена одним проектом. Например, нельзя добавлять в диаграмму классов типы, определенные в разных проектах, даже если они являются частями одного и того же решения.

Надстройку **Class Designer** можно разделить на четыре компонента:

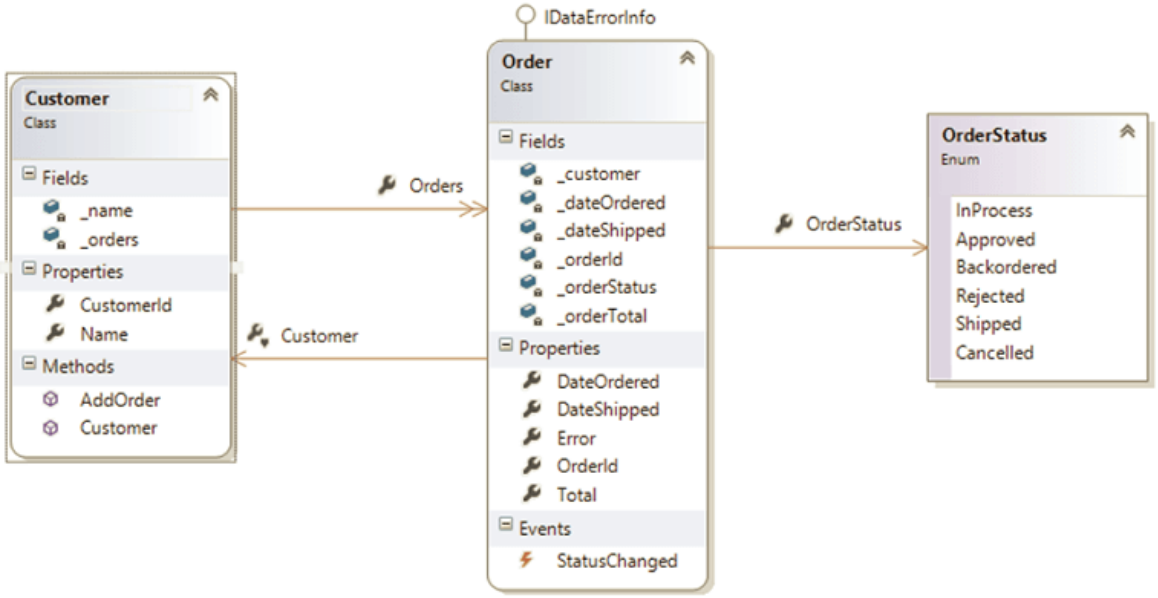
- область проектирования (design surface),
- инструментальная панель Toolbox,
- окно Class Details,
- сетка свойств.

Изменения диаграммы классов сохраняются в файле с расширением *.cd, который в сочетании с исходными файлами используется для генерации схемы, демонстрируемой в области проектирования Class Designer.

Область проектирования диаграммы классов

Область проектирования надстройки Class Designer позволяет разработчику взаимодействовать с типами, используя перетаскивание компонентов. Разработчик может добавлять в область проектирования существующие типы, перетаскивая их из любого представления классов или окна Solution Explorer. Если файл, связанный с окном Solution Explorer, содержит несколько типов, то все они добавляются в область проектирования.

На рисунке ниже показана простая диаграмма классов, содержащая два класса: Customer и Order, а также перечисление OrderStatus. Каждый класс содержит поля, свойства, методы и события. Между классами существует ассоциация, поскольку класс Customer содержит свойство Orders, представляющее собой список объектов типа Order, а класс Order реализует интерфейс IDataErrorInfo. Вся эта информация содержится на диаграмме классов.



Каждый класс на диаграмме классов изображается как отдельная сущность, которую можно перетаскивать по области проектирования и при необходимости изменять размеры. Класс состоит из полей, свойств, методов и событий. На рисунке выше эти компоненты сгруппированы в разделы. Разработчик может изменять шаблон для диаграмм классов, например, перечислять компоненты в алфавитном порядке или группировать их по уровню доступа.

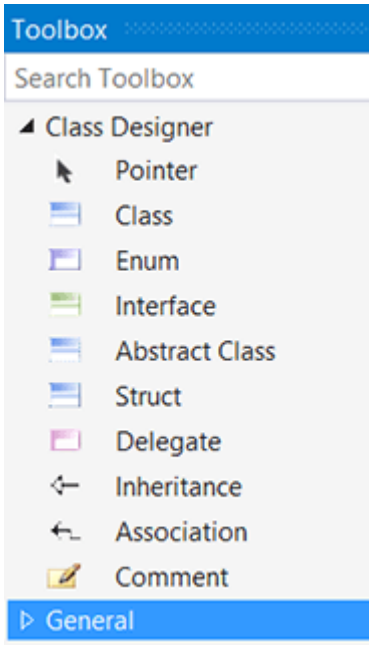
Надстройка Class Designer часто используется для просмотра многочисленных классов, чтобы было понятно, как они связаны друг с другом. В таком случае удобно скрывать компоненты классов, чтобы упростить диаграмму. Для того чтобы скрыть все компоненты одновременно, можно использовать переключатель, расположенный в правом верхнем углу представления класса в области проектирования. Если необходимо скрыть не все, а

лишь некоторые компоненты, то их можно скрывать по отдельности или по разделам, щелкая правой кнопкой мыши на соответствующем элементе и выбирая в контекстном меню команду Hide.

Инструментальная панель Toolbox

Чтобы создать элемент, следует перетащить его из инструментальной панели Toolbox в область проектирования и дважды щелкнуть на нем. На рисунке ниже показана инструментальная панель Toolbox с видимой закладкой Class Designer.

Элементы на панели Toolbox можно разделить на *сущности (entities)* или *соединители (connectors)*. Обратите внимание на то, что элемент Comment можно добавить на закладку Class Designer, но он не появляется ни в одной части кода, а просто добавляется в документацию о диаграмме классов.



Сущности (entities)

Сущности, которые можно добавить на диаграмму классов, полностью соответствуют типам в технологии .NET. Добавляя новую сущность в область проектирования, необходимо присвоить ей имя. Кроме того, требуется указать, в какой файл ее следует включить: новый или существующий.

Для того чтобы удалить сущности из диаграммы, надо щелкнуть на них правой кнопкой мыши и выбрать в контекстном меню *команду Remove From Diagram*. В этом случае они не удаляются из исходного кода, а исчезают из диаграммы классов. Если разработчик все же хочет удалить связанный с сущностью исходный код, следует выбрать *команду Delete Code*.

Для того чтобы увидеть код, связанный с сущностью, следует дважды щелкнуть на сущности или выбрать команду View Code в контекстном меню, которое открывается после щелчка правой кнопкой мыши. Сущности, включенные в инструментальную панель Toolbox, описаны ниже.

Class (класс)
Поля, свойства, методы, события и константы можно добавлять в класс с помощью контекстного меню, открывающегося после щелчка правой кнопкой мыши или с помощью окна Class Details. Несмотря на то, что класс может поддерживать вложенные типы, они не могут добавляться в область проектирования. Классы могут также реализовывать интерфейсы. Например, на рисунке выше класс Order реализует интерфейс IDataErrorInfo.

Enum (перечисление)
Может содержать только список членов с присвоенными им значениями. Каждый метод имеет резюме и комментарии, но они появляются только в виде XML-комментария, расположенного напротив метода.

Interface (интерфейс)
Интерфейсы определяют свойства, методы и события, которые должен реализовать класс. Интерфейсы могут также содержать вложенные типы, но напомним, что добавление вложенных типов не поддерживается утилитой Class Designer.

Abstract Class (абстрактный класс)
Абстрактные классы не отличаются от обычных классов, за исключением того, что их имена в области проектирования набираются курсивом и снабжаются меткой Abstract (в языке C#) или Must Inherit (в языке VB).

Struct (структура)
Это единственная сущность, кроме перечислений, которая появляется в области проектирования в прямоугольнике. Аналогично классу, структура имеет поля, свойства, методы, события и константы. Кроме того, она тоже может содержать вложенные типы. Тем не менее, в отличие от класса структура не имеет деструктора.

Delegate (делегат)
Несмотря на то, что делегат появляется на диаграмме классов как сущность, он не может содержать вложенные типы. Он может содержать только параметры, определяющие его сигнатуру.

Соединители (connectors)
Между сущностями могут быть установлены отношения двух типов. Они иллюстрируются на диаграмме классов с помощью соединителей, описанных ниже:

Inheritance (Наследование)
Соединитель Inheritance используется для того, чтобы показать отношение между классами, наследующими свойства друг друга.

Association (Ассоциация)
Если класс содержит ссылку на другой класс, то между ними существует ассоциация. Она демонстрируется с помощью соединителя Association. Если отношение основано на коллекции, - например, на списке объектов типа Order, - его можно представить с помощью *ассоциации коллекции (collection association)*. Ассоциация коллекции Orders показана на рисунке выше в виде соединителя, связывающего классы Customer и Order.

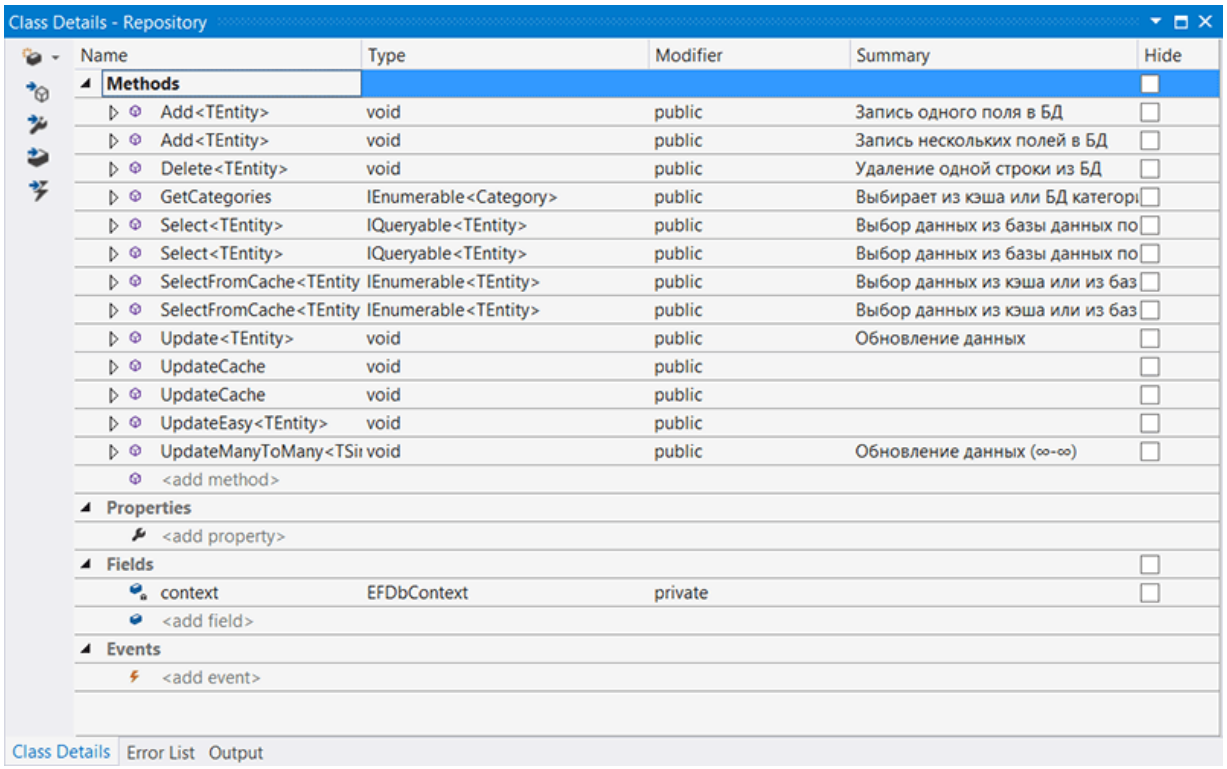
Ассоциацию классов (class association) можно представить либо в виде поля или свойства класса, либо в виде ассоциативной связи между классами. Для этого можно использовать контекстное меню, открывающееся после щелчка правой кнопкой мыши на поле, свойстве или линии связи между классами.

Для того чтобы показать свойство как ассоциацию коллекции, необходимо щелкнуть правой кнопкой мыши на классе и выбрать *команду Show as Collection Association*. В результате свойство будет скрыто в классе и показано в виде соединителя с ассоциированным классом на диаграмме.

Окно Class Details

Компонент можно добавить в сущность, щелкнув правой кнопкой мыши и выбрав нужный компонент. К сожалению, это довольно долгий процесс, не позволяющий добавлять параметры метода или возвращаемые значения.

Надстройка Class Designer в системе Visual Studio содержит окно Class Details, обеспечивающее пользовательский интерфейс, позволяющий быстро вставлять компоненты. Это окно показано на рисунке ниже на примере класса репозитория для работы с базой данных через Entity Framework:



В левой части окна расположены кнопки, которые могут помочь в управлении классами, содержащими большое количество компонентов. Верхнюю кнопку можно использовать для того, чтобы добавлять в класс методы, свойства, поля или события. Остальные кнопки предназначены для перемещения фокуса по группам. Например, вторая кнопка позволяет перейти к списку методов класса. Перемещаться по этому списку можно с помощью клавиш со стрелками, направленными вверх и вниз.

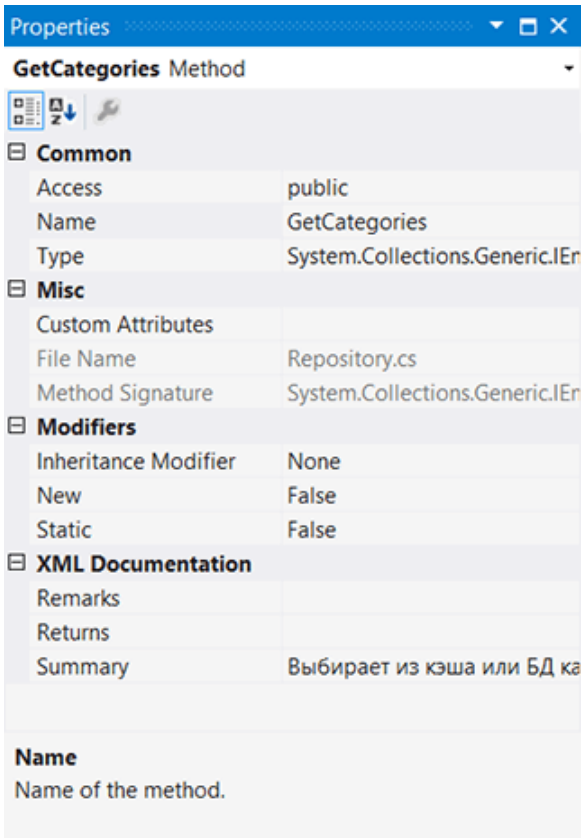
Как показано на рисунке, основная область окна разделяется на четыре списка, упорядоченных по алфавиту: Methods, Properties, Fields и Events. Сущности других типов могут содержать другие компоненты, например Members и Parameters. Каждая строка разделяется на пять столбцов, содержащих имя, тип возвращаемого значения, уровень

доступа, описание, а также индикатор того, скрыт ли данный элемент в области проектирования. В каждом случае поле Summary представляет собой XML-комментарий, расположенный напротив соответствующего компонента. События отличаются от других компонентов тем, что в столбце Type должен быт указан делегат. Перемещаться по столбцам можно с помощью клавиш со стрелками, направленными влево и вправо, клавиши <Tab> (следующий столбец) и комбинации клавиш <Shift+Tab> (предыдущий столбец).

Для того чтобы ввести параметры метода, следует с помощью клавиши со стрелкой, направленной вправо, раскрыть узел метода, чтобы появился список параметров. Выбрав узел Add Parameter, пользователь может добавить новый параметр метода. После этого новый параметр можно найти, перемещаясь с помощью клавиш со стрелками.

Окно Properties

Несмотря на полезность окна Class Details, оно не предоставляет полной информации о сущностях, которыми являются компоненты. Например, свойства могут быть предназначены только для чтения, но этот факт никак не отображается в окне Class Details. Окно Properties, показанное на рисунке ниже, содержит полный список атрибутов метода GetCategoies() класса репозитория:



На рисунке показано, что метод GetCategoies() является открытым и не является статическим. Кроме того, в окне показано, что этот метод определен в классе Repository.cs. Класс, состоящий из частичных классов, можно разделить между несколькими файлами. Когда пользователь выбирает частичный класс, свойство File Name показывает все файлы, определяющие этот класс, в виде списка, в котором элементы разделены запятыми. Несмотря на то, что в этом окне некоторые свойства предназначены только для чтения, их можно уточнять в соответствующем исходном файле.

Компоновка
Поскольку диаграмма классов предназначена для визуализации классов и отношений между ними, система Visual Studio предоставляет разработчику несколько инструментальных панелей с элементами управления, позволяющими компоновать сущности в области проектирования. На рисунке ниже показана инструментальная панель, являющаяся частью области проектирования.

Первые три кнопки управляют размером элементов на области проектирования, позволяя увеличивать и уменьшать масштаб сущностей. Следующая кнопка используется для отображения подробных сведений класса окна. Следующие три кнопки позволяют изменить расположение компонентов сущностей: слева направо, кнопки группировки по виду, группировки по доступу и сортировки в алфавитном порядке. Следующие две кнопки используются для автоматизации процесса размещения сущностей на поверхности конструкции: слева находится кнопка схемы расположения, которая автоматически перемещает объекты по поверхности проектирования, также сворачивая сущности, скрывая все компоненты; правая кнопка регулирует размер субъектов таким образом, чтобы все компоненты полностью были открыты.

Стиль отображения сущностей можно настроить с помощью трех кнопок. Левая кнопка - Display Name - задает стиль отображения, при котором демонстрируется только имя компонента. Этот стиль можно расширить, используя кнопку Display Name and Type так, чтобы демонстрировались и имя, и тип компонента. Часто это оказывается удобнее, хотя и занимает больше места на экране.

Экспорт диаграмм
Довольно часто процесс выбора классов для архитектуры системы является частью более крупного процесса проектирования или анализа. По этой причине нередко возникает необходимость экспортировать диаграмму класса для ее включения в отчеты.

Экспортировать диаграмму классов можно либо щелчком правой кнопки мыши в любом месте области проектирования, либо с помощью меню Class Diagram, в любом случае, выбрав *команду Export Diagram as Image menu*, разработчик откроет диалоговое окно, позволяющее выбирать формат изображения и задавать имя файла для сохранения диаграммы.

Изображение можно также непосредственно скопировать и вставить в документ Microsoft Word или графический редактор, например Visio. Для этого необходимо сначала выбрать один или несколько классов на диаграмме. Кроме того, диаграммы классов можно печатать непосредственно из системы Visual Studio с помощью команды File --> Print.

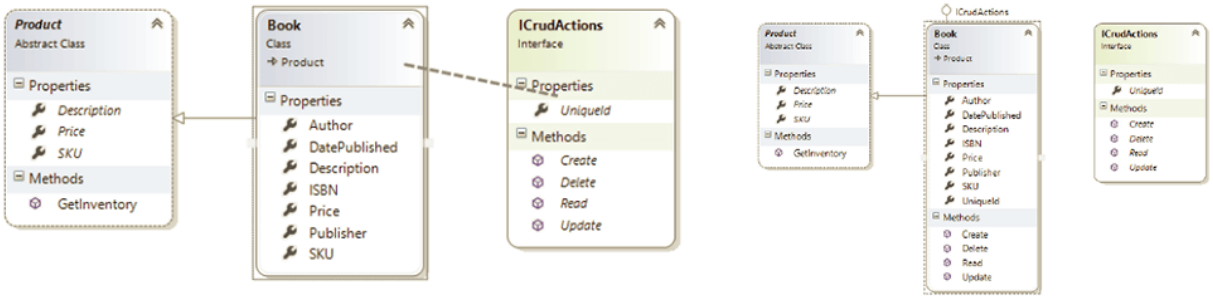
Одна из главных целей системы Visual Studio и технологии .NET - уменьшить объем кода, который должны писать разработчики. Эта цель достигается двумя путями: либо уменьшается общий объем кода, который должен быть написан, либо уменьшается объем кода, который должен быть написан вручную. Первый подход основан на очень богатом наборе базовых классов, включенных в каркас .NET. Второй подход, нацеленный на уменьшение кода, который должен быть написан вручную, поддерживается средствами генерации кода и рефакторинга, включенными в утилиту Class Designer.

Генерация кода методом drag-and-drop

Почти каждое действие, выполненное над диаграммой классов, приводит к изменению исходного кода и, по существу, представляет собой определенный уровень генерации кода.

Однако некоторые более сложные действия, связанные с генерацией кода, можно выполнить, манипулируя диаграммой классов. Как уже упоминалось ранее, с помощью соединителя Inheritance можно установить отношение наследования между базовым и производным классами. При этом файл кода производного класса обновляется, отражая внесенные изменения. Но если базовый класс является абстрактным, как, например, класс Product на рисунке ниже, то надстройка Class Designer может выполнить дополнительный анализ и генерацию кода. Если базовый класс является абстрактным и содержит абстрактные члены, то они автоматически реализуются в производных классах. Это показано на рисунке справа, где абстрактные свойства Description, Price и SKU были добавлены в класс Book. Метод GetInventory() не был реализован, потому что не был помечен как абстрактный.

Соединитель Inheritance можно использовать еще в одной операции, которая приводит к автоматической генерации кода. На рисунке ниже, слева, в диаграмму был добавлен интерфейс ICrudActions. Когда соединитель Inheritance перетаскивается из класса в интерфейс, все члены интерфейса реализуются в классе:



Следующий пример содержит код, сгенерированный, когда интерфейс ICrudActions был добавлен в класс Book:

```
#region ICrudActions Members
public Guid UniqueId
{
    get
    {
        throw new NotImplementedException();
    }
    set
    {
        throw new NotImplementedException();
    }
}
```

```
public void Create()
{
    throw new NotImplementedException();
}

public void Update()
{
    throw new NotImplementedException();
}

public void Read()
{
    throw new NotImplementedException();
}

public void Delete()
{
    throw new NotImplementedException();
}
#endregion
```

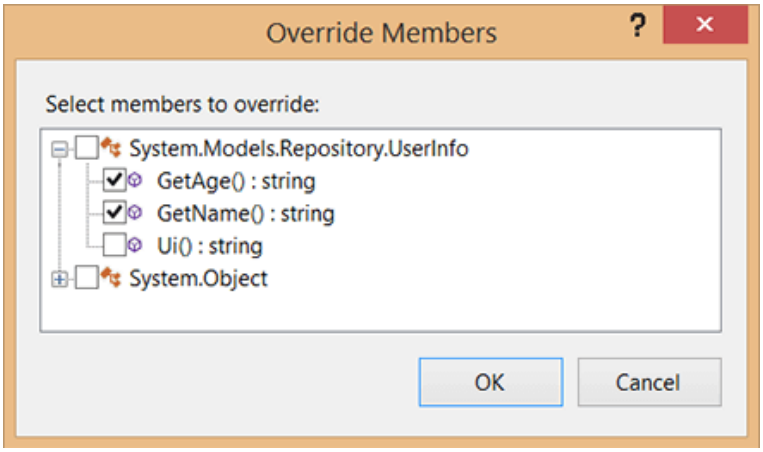
Генерация кода с помощью технологии IntelliSense

Доступ к остальным функциям, связанным с генерацией кода с помощью надстройки Class Designer, обеспечивает подменю, имеющее довольно странное имя IntelliSense. Поскольку эти функции применяются только к классам, это меню становится видимым только тогда, когда разработчик выбирает какой-нибудь обычный или абстрактный класс на диаграмме.

В этом меню доступны две функции генерации кода: Implement Abstract Class и Override Members.

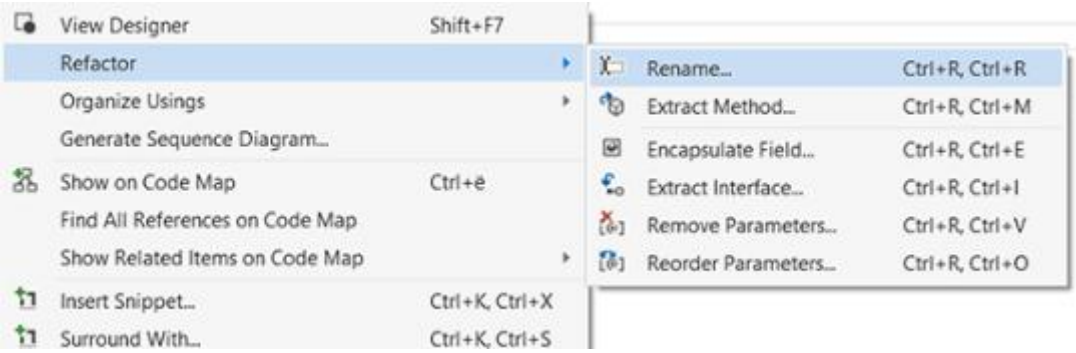
Функция Implement Abstract Class гарантирует, что все абстрактные члены из базового класса будут реализованы в производном классе. Для того чтобы получить доступ к этому классу, необходимо щелкнуть правой кнопкой мыши на производном классе, выбрать команду IntelliSense, а затем команду Implement Abstract Class.

Подобное делает и *функция Override Members*, которая используется для того, чтобы выбирать подлежащие замещению открытые свойства или методы в базовом классе. Для доступа к этой функции необходимо щелкнуть правой кнопкой мыши на производном классе, выбрать команду IntelliSense, а затем команду Override Members. Диалоговое окно, показанное на рисунке ниже, содержит базовые классы, а также свойства и методы, которые еще не были замещены.



Рефакторинг с помощью надстройки Class Designer
Система Visual Studio поддерживает рефакторинг кода с помощью окна редактирования кода. Надстройка Class Designer тоже имеет много функций, связанных с рефакторингом при работе с сущностями на диаграмме классов.

Для того чтобы получить доступ к функции рефакторинга в надстройке Class Designer, необходимо щелкнуть правой кнопкой мыши на сущности или на любом из ее членов и выбрать команду из меню Refactor.



В этом меню доступны следующие функции рефакторинга:

Rename Types and Type Members
Переименовывает тип или его член на диаграмме классов или в окне Properties. Переименование типа или его члена приводит к изменениям всех фрагментов кода, где ранее встречалось его старое имя. Можно даже убедиться, что эти изменения распространяются также на все комментарии и статические строки.

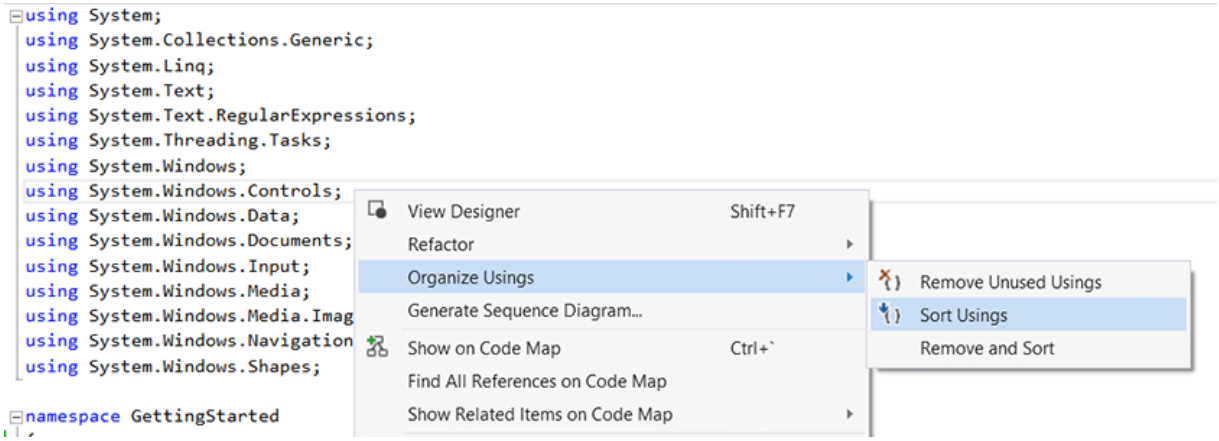
Encapsulate Field
Быстро создает новое свойство на основе существующего поля и аккуратно обновляет код, добавляя в него ссылки на новое свойство.

Reorder or Remove Parameters (только для языка C#)
Изменяет порядок следования параметров метода в типе или удаляет параметр из метода.

Extract Interface (только в языке C#)
Разработчик может извлекать члены типа в новый интерфейс. Эта функция позволяет выбирать только то подмножество членов, которое необходимо выделить в новый интерфейс.

Для того чтобы копировать и перемещать члены между типами, можно также использовать стандартные операции Cut, Copy и Paste, предусмотренные в системе Windows.

Меню Organize Usings
Очень полезно поддерживать упорядоченный список директив using в каждом файле (на языке C#) и ссылаться только на те пространства имен, которые действительно необходимы в данном файле. Команда Organize Usings, доступная в контекстном меню, которое открывается после щелчка пользователем правой кнопкой мыши в окне редактора кода, поможет в обоих этих случаях:



После рефакторинга кода может выясниться, что в начале файла содержится множество директив using, которые больше не используются. Чтобы определить, какие из этих директив используются, а какие нет, вместо метода проб и ошибок можно выполнить операцию, предусмотренную системой Visual Studio. Для этого достаточно щелкнуть правой кнопкой мыши в окне редактора кода и выбрать команду Organize Usings --> *Remove Unused Usings* (в языке C#). Неиспользуемые директивы using, их альтернативные имена и внешние альтернативные имена сборок из исходного файла будут удалены.

Очень полезно упорядочивать директивы using в алфавитном порядке, чтобы легко управлять пространствами имен, на которые они ссылаются. Для того чтобы не делать это вручную, нужно щелкнуть правой кнопкой мыши в окне редактора кода и выбрать команду Organize Usings --> *Sort Usings*.

Для того чтобы одновременно упорядочить директивы using и удалить неиспользуемые, достаточно щелкнуть правой кнопкой мыши в окне редактора кода и выбрать команду Organize Usings --> *Remove and Sort*.

Задание:

1. **Создайте новый проект**

Создайте консольное приложение на C# (.NET) с именем ClassDiagramDemo.

2. **Добавьте диаграмму классов**

Щелкните правой кнопкой мыши по проекту в **Solution Explorer** → выберите **Add** → **New Item** → найдите и добавьте шаблон **Class Diagram** (файл с расширением .cd).

3. **Создайте диаграмму классов вручную**

Используя **Toolbox** → **Class Designer**, добавьте на диаграмму следующие сущности:

- **Абстрактный класс** Product (с именем Product, пометьте как Abstract)

Добавьте **абстрактные** члены:

- string Name (свойство)
- decimal Price (свойство)
- string SKU (свойство)

Добавьте **обычный** метод: public void DisplayInfo() - не абстрактный, тело не нужно.

- **Класс** Book - наследует от Product

- **Интерфейс** ICrudActions

Добавьте в него 5 абстрактных членов:

- Guid UniqueId (свойство)
- void Create()

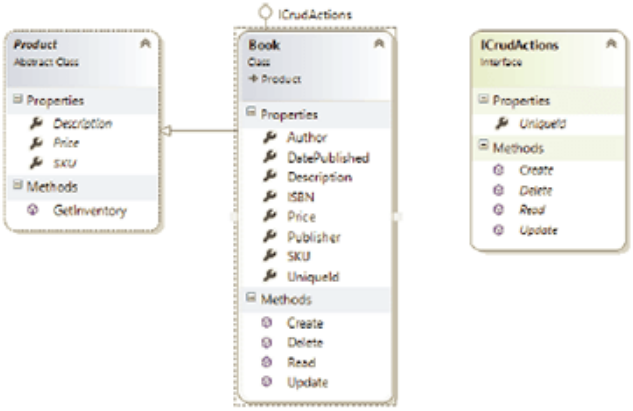
- void Update()
- void Read()
- void Delete()

- Связи:
- Установите **наследование** (Inheritance) от Product к Book
- Установите **наследование** (Inheritance) от ICrudActions к Book

4. **Сгенерируйте код автоматически**

Щелкните правой кнопкой мыши по классу Book → выберите **IntelliSense** → **Implement Abstract Class** → Обратите внимание: в файле Book.cs автоматически сгенерированы реализации всех абстрактных членов Product.

Щелкните правой кнопкой мыши по классу Book → выберите **IntelliSense** → **Implement Interface** → Обратите внимание: в файле Book.cs автоматически сгенерированы все члены интерфейса ICrudActions (с throw new NotImplementedException();).



5. **Просмотрите сгенерированный код**

Откройте файл Book.cs в редакторе кода.

6. **Запишите в отчет**:
- Какие члены были сгенерированы автоматически?
- Какие из них были добавлены как результат наследования от Product?
- Какие - как результат реализации ICrudActions?
- Сравните с диаграммой: совпадают ли элементы?

7. **Рефакторинг:**

В диаграмме щелкните правой кнопкой мыши по свойству Name в классе Product → выберите **Refactor** → **Rename Types and Type Members** → Переименуйте его в Title. Обратите внимание: в файле Product.cs и Book.cs все упоминания Name автоматически заменились на Title.

Теперь в классе Book найдите поле UniqueId (сгенерированное из интерфейса). Щелкните по нему правой кнопкой → **Refactor** → **Encapsulate Field**→ Создайте свойство UniqueId на основе этого поля (если поле не существует - создайте его предварительно вручную в коде, если Class Designer не отображает его как поле). Убедитесь, что все обращения к полю заменились на обращение к свойству.

8. **Экспорт диаграммы**

Сохраните диаграмму. Экспортируйте её как изображение (Export Diagram as Image) в формате PNG и вставьте в отчет.

В итоге должна получиться программа:

Класс Product:

```
using System;

namespace ClassDiagramDemo
{
    public abstract class Product
    {
        public string Title { get; set; } // Переименовано из Name → Title через Refactor
        public decimal Price { get; set; }
        public string SKU { get; set; }

        public void DisplayInfo()
        {
            Console.WriteLine($"Product: {Title}, SKU: {SKU}, Price: {Price:C}");
        }
    }
}
```

Интерфейс ICrudActions:

```
namespace ClassDiagramDemo
{
    public interface ICrudActions
    {
        Guid UniqueId { get; set; }
        void Create();
        void Update();
        void Read();
        void Delete();
    }
}
```

Класс Book:

```
using System;

namespace ClassDiagramDemo
{
    public class Book : Product, ICrudActions
    {
        // Реализация интерфейса ICrudActions
        public Guid UniqueId { get; set; } = Guid.NewGuid(); // Инициализировано для избежания
null

        public void Create()
        {
            Console.WriteLine($"Book '{Title}' created with ID: {UniqueId}");
        }

        public void Update()
        {
            Console.WriteLine($"Book '{Title}' with ID {UniqueId} updated.");
        }

        public void Read()
        {
            Console.WriteLine($"Reading book: {Title} (ID: {UniqueId})");
        }

        public void Delete()
        {
            Console.WriteLine($"Book '{Title}' with ID {UniqueId} deleted.");
        }
```

```
        // Дополнительный метод для удобства
        public void DisplayBookInfo()
        {
            DisplayInfo(); // Наследуемый метод
            Console.WriteLine($"Book Unique ID: {UniqueId}");
        }
    }
}
```

Основная программа:

```
using System;

namespace ClassDiagramDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            // Создаем экземпляр Book
            Book book = new Book
            {
                Title = "C# 12 in Depth",
                Price = 5000.00m,
                SKU = "ISBN-978-1-234567-89-0"
            };

            // Тестируем методы
            book.DisplayBookInfo();
            Console.WriteLine();

            // Имитируем работу с объектом
            book.Create();
            book.Read();
            book.Update();
            book.Delete();
        }
    }
}
```